

《软件体系结构》2026 期末考试高分复习指南

适用对象：南京大学软件学院研究生 / 大三本科生
考试形式：闭卷英文题面，可用中文作答 | 100分
整理依据：2026真实考试回忆卷 + 课程课件（ADD 3.0、架构模式、质量属性）+ 多源复习笔记

🔥 使用建议：本指南按照2026年真实考试题型和知识点组织，覆盖全部考点。每个知识点包含「概念定义 → 关系/区别 → 例子或权衡」三层结构，可直接用于答题。

目录

- [Part 0: 核心思维框架 —— 架构的本质](#)
- [Part I: 简答题 \(10题×6分=60分\)](#)
 - [QI-1: 需求、质量属性与ASR](#)
 - [QI-2: 6个通用架构设计策略](#)
 - [QI-3: 质量属性场景建模 \(可用性 & 可修改性\)](#)
 - [QI-4: 架构模式与战术的区别和关系](#)
 - [QI-5: 架构设计主要过程步骤 \(ADD\)](#)
 - [QI-6: DDD中聚合模式的价值](#)
 - [QI-7: 为什么需要不同视图 + 4个示例视图](#)
 - [QI-8: 微服务系统的4个架构元素](#)
 - [QI-9: 事件驱动架构风格](#)
 - [QI-10: 企业架构中的4A](#)
- [Part II: 分析论述题 \(2题×10分=20分\)](#)
 - [QII-1: 7类设计决定与Binding Time](#)
 - [QII-2: C/S → 分层/三层 → SOA的演进](#)
- [Part III: 设计题 \(20分\)](#)
 - [DDD事件风暴 + 聚合设计方法](#)
- [附录: 质量属性速查表](#)

Part 0: 核心思维框架 —— 架构的本质

0.1 什么是软件架构?

官方定义（《Software Architecture in Practice》）：
软件架构是用于推理系统的一组结构（structures），这些结构包含软件元素（elements）、它们之间的关系（relations），以及两者的属性（properties）。

我的理解：架构不是画几张盒式图就完事了。架构是最难更改的那些设计决策的集合。它之所以重要，是因为：

1. 架构是早期设计决策的体现 — 这些决策一旦做出，改动的代价极高
2. 架构促进/阻碍质量属性的实现 — 性能、安全性、可修改性等都在架构层面被锁定
3. 架构是涉众沟通的载体 — 不同角色（开发、测试、运维、管理）通过架构理解系统
4. 架构是可迁移和可重用的抽象 — 一种架构可以映射到多个系统

 **关键区分：**所有架构都是设计，但并非所有设计都是架构。架构是**更高层的设计**，关注的是结构、组件之间的接口和交互，而不是实现细节。

0.2 架构不是什么

 错误理解	 正确理解
架构就是盒式线图（Box-and-line drawings）	盒式图只是起点，架构还包含 组件行为
架构是固定的、一次性的	架构随系统演化而变化
架构只关心技术	架构服务于 业务目标和质量属性
架构越新越好	架构选择的本质是在特定时代约束下对质量属性做有意识的优先级排序

0.3 本课程的核心主线：架构的演进逻辑

架构演进不是"替代史"，而是"主要矛盾迁移史"

算力稀缺 → 多用户共享 → 富交互 → 系统内分层 → 企业级协同 → 团队自治发布 → 平台化韧性 → 语义增强 → 认知式执行

每一代架构都在回应当时最紧迫的技术与业务矛盾。真正要学的是"迁移逻辑"——知道旧模式为什么失效，才知道新模式为什么出现。这不是记忆术语的课，而是训练"看懂时代矛盾并做边界取舍"的能力。

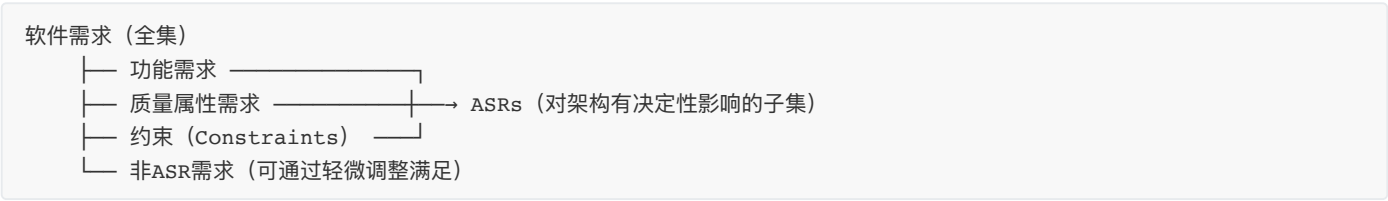
Part I：简答题（10题×6分=60分）

QI-1 需求、质量属性与架构显著需求（ASR）

概念定义

概念	英文	定义
软件需求	Software Requirements	系统必须做什么的陈述，包括功能需求和非功能需求
质量属性	Quality Attributes (QA)	系统在其功能之上应提供的 可度量特性 ，如性能、可用性、可修改性、安全性等
架构显著需求	Architecturally Significant Requirements (ASRs)	需求的一个 子集 ，是那些对架构设计有 决定性影响 的需求

区别与关系



核心关系：

- 1. **ASR是需求的子集：**不是所有需求都是ASR。ASR是那些**塑造架构形状**的需求
- 2. **质量属性是ASR的主要来源：**质量属性需求（如"系统必须在30秒内恢复"）往往成为ASR
- 3. **功能需求也可以成为ASR：**当功能涉及核心业务逻辑且影响系统分解时

4. 约束天然**是ASR**：约束是自由度为0的设计决策（如"必须使用Java"、"必须部署在AWS上"）

 **我的理解**：ASR本质上是在问"什么需求会决定架构的骨架？"比如一个选课系统，"支持5万并发选课"是ASR（决定你用什么部署方案），而"教务员能导出Excel报表"大概率不是（加个导出模块就行，不影响整体架构）。

答题模板：

- 1. 定义三个概念
- 2. 画关系图（需求是全集，ASR是关键子集，质量属性是ASR主要来源）
- 3. 举例：以选课系统为例，说明哪些是ASR、哪些不是

QI-2 六个通用架构设计策略

六大策略

#	策略	英文	核心理念	例子
1	分解	Decomposition	将大系统拆分为可管理的小部分，分而治之	将电商系统拆为用户、订单、商品、支付等模块
2	抽象	Abstraction	提取共性、隐藏细节，通过接口隔离变化	定义 <code>PaymentGateway</code> 接口，隐藏支付宝/微信支付的实现差异
3	分层/分离关注点	Separation of Concerns	不同职责分到不同层/模块，每层只关注自己的逻辑	MVC模式：展示层、业务层、数据层各司其职
4	泛化/通用化	Generalization	设计不受不必要限制的通用方案，支持复用	设计用户权限系统时不仅支持当前角色，预留自定义角色扩展点
5	增量开发	Incremental Development	以小增量逐步构建系统，每步产出可工作的中间版本	先做MVP（最小可行产品），再迭代增加高级功能
6	预期变化	Anticipation of Change	识别可能的变更点，通过设计使变更影响局部化	使用策略模式封装可能变化的算法，使用配置文件管理可变参数

 **我的理解**：这六大策略不是独立的，而是相互配合的。分解时必须配合抽象（否则只是混乱地切分），分层本身就是一种分离关注点的形式，泛化是为了让系统更好地预期变化。实际架构设计中，这些策略同时起作用。

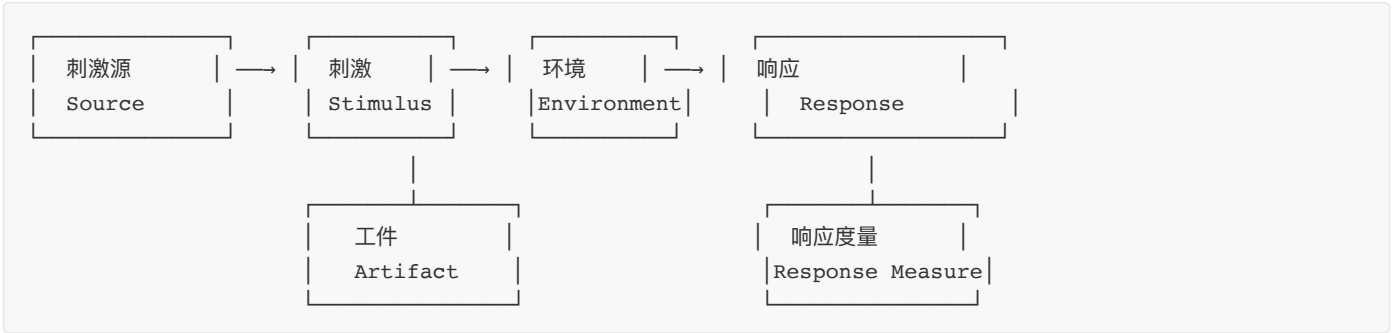
答题模板：

- 1. 列出6个策略，每个配一句定义
- 2. 选2-3个展开，结合具体例子（如选课系统或电商系统）
- 3. 强调这些策略是相互配合的

QI-3 质量属性场景建模（可用性 & 可修改性）

质量属性场景的6要素

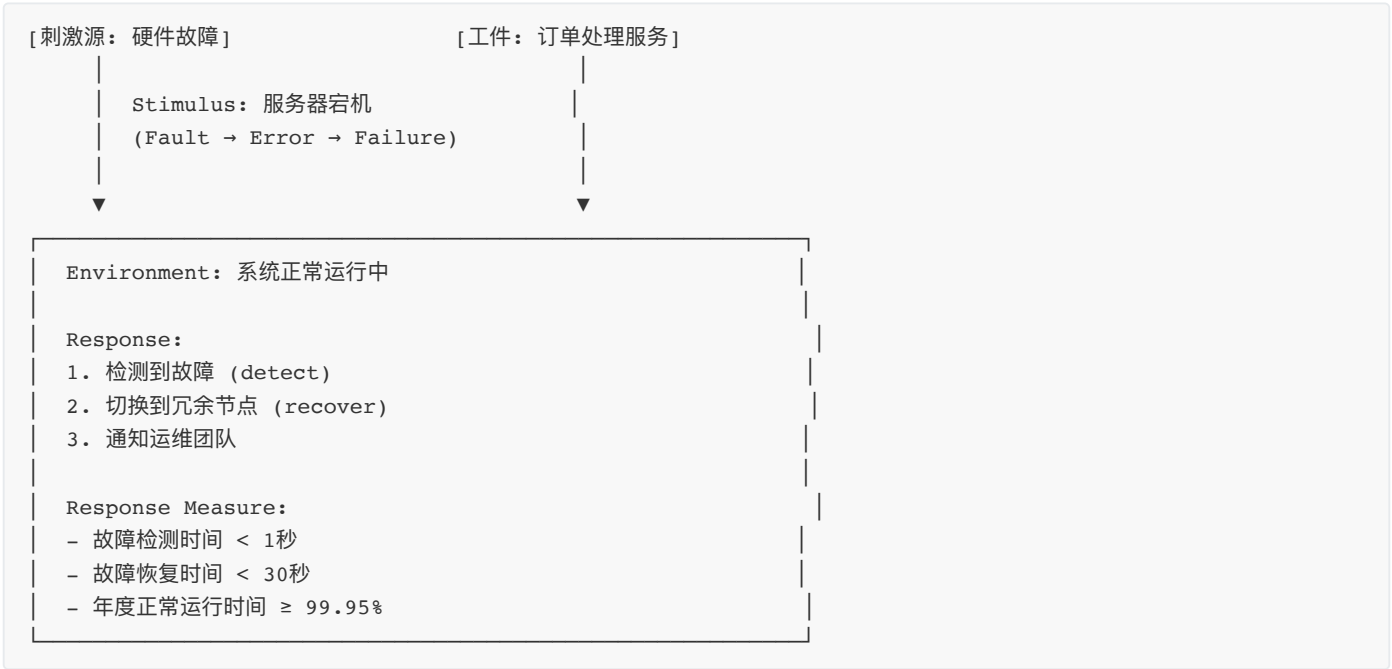
质量属性场景（Quality Attribute Scenario）是精确描述一个质量属性需求的**6元组**框架：



要素	英文	说明	例子（可用性）
刺激源	Source	产生刺激的实体（人、系统、传感器等）	外部监控系统
刺激	Stimulus	到达系统需要考虑的条件	心跳信号丢失（检测到故障）
环境	Environment	发生刺激时系统的状态	正常运行状态
工件	Artifact	被影响的系统部分	核心业务进程
响应	Response	刺激到来后的活动	切换到备用节点，继续提供服务
响应度量	Response Measure	可量化的测量标准	故障切换时间 < 30秒；服务可用性 ≥ 99.95%

可用性（Availability）质量属性场景

刺激-响应图：



可用性关键公式：

$Availability = \frac{MTBF}{MTBF+MTTR}$

其中 MTBF = Mean Time Between Failures（平均无故障时间），MTTR = Mean Time To Repair（平均修复时间）。

我的理解：可用性战术的核心思路是 "让不可用时间尽可能短"。所有可用性策略本质上都在做三件事中的一件或多件：

- 减少故障发生频率（提高 MTBF）
- 缩短故障检测时间
- 缩短故障恢复时间（降低 MTTR）

可用性战术树（核心记忆框架）：



三种冗余的对比：

冗余方式	切换时间	资源开销	适合场景
Active Redundancy	最快（毫秒级）	最高（所有节点满负荷）	金融交易系统、电信核心网
Passive Redundancy	较快（秒级）	中等（备节点待机）	Web服务器、应用服务器
Spare	慢（分钟级）	最低（冷备资源池）	非关键业务、批处理系统

可修改性（Modifiability）质量属性场景

刺激-响应图：



可修改性战术：

可修改性战术

- └─ 减小模块尺寸 (Reduce Size of Module)
 - └─ 拆分模块 (Split Module)
- └─ 增加内聚 (Increase Cohesion)
 - └─ 将相关的职责聚合到一起
- └─ 减少耦合 (Reduce Coupling)
 - └─ 封装 (Encapsulation)
 - └─ 使用中介者 (Use an Intermediary)
 - └─ 抽象通用服务 (Abstract Common Services)
 - └─ 依赖倒置 (Dependency Inversion)
- └─ 推迟绑定 (Defer Binding)
 - └─ 运行时注册/发现 (Runtime Registration)
 - └─ 配置文件驱动 (Configuration Files)
 - └─ 多态/接口编程 (Polymorphism)
 - └─ 依赖注入 (Dependency Injection)

 我的理解：可修改性的本质是 "让变化的影响范围尽量局部化"。所有可修改性战术都在做一件事：将变化点隔离在最小的边界内。好的架构不是"永不修改"，而是"改了A不影响B"。

答题模板：

- 1. 先说明6要素框架
- 2. 分别画出可用性和可修改性的刺激-响应图
- 3. 每个场景写清楚6要素，特别注意响应度量必须是可量化的

QI-4 架构模式与战术的区别和关系

概念定义

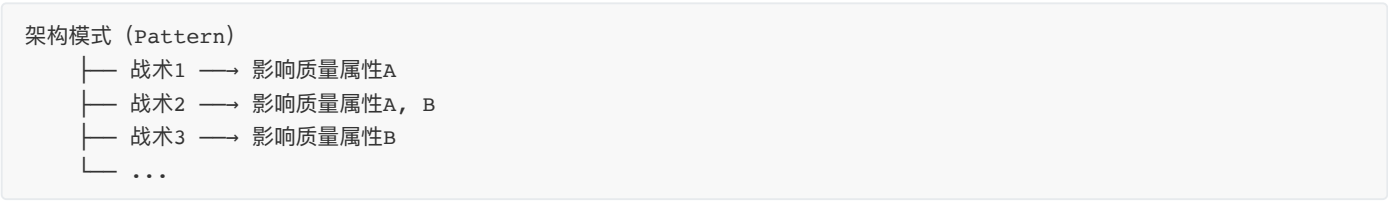
概念	英文	定义
架构战术	Tactic	单一的设计决策，影响某个质量属性响应的控制。是原子级的设计手段
架构模式	Pattern	对反复出现的设计问题的经过验证的概念性解决方案。由多个战术组成

区别

维度	战术 (Tactic)	模式 (Pattern)
粒度	原子级，单一设计决策	组合级，一组设计决策的集合
关注点	通常只影响一个质量属性	影响多个质量属性，做跨属性的权衡
抽象层级	较低，更接近实现	较高，更接近架构骨架
可组合性	可以被其他战术组合	内部已包含了多个战术
例子	冗余 (Redundancy)、缓存 (Caching)、加密 (Encryption)	分层模式 (Layered)、微服务模式、事件驱动模式

关系

模式是由战术构成的，战术是模式的"零件"。



两个具体模式及其关联战术

例1：分层模式 (Layered Pattern)

包含的战术	影响的质量属性
增加语义内聚 (Increase Semantic Cohesion)	可修改性 ↑
封装 (Encapsulation)	可修改性 ↑
使用中介者 (Use an Intermediary)	可修改性 ↑
抽象通用服务 (Abstract Common Services)	可修改性 ↑、可复用性 ↑

但代价是：增加层间通信延迟 → 性能↓

例2：事件驱动模式 (Event-Driven Pattern)

包含的战术	影响的质量属性
异步通信	可用性 ↑、弹性 ↑
消息缓冲/队列	性能（削峰）↑
发布-订阅解耦	可修改性 ↑、可扩展性 ↑
事件溯源	可审计性 ↑、可恢复性 ↑

但代价是：时序推理变难 → 可调试性↓，最终一致性 → 强一致性↓

 **我的理解：**可以把战术类比为"设计技巧"（比如用缓存来提速），把模式类比为"设计方案模板"（比如整个系统用微服务架构）。模式告诉你整体的组织方式，战术告诉你具体的实现手段。在实际设计中，通常**先选模式（确定骨架），再用战术（细化实现）**。

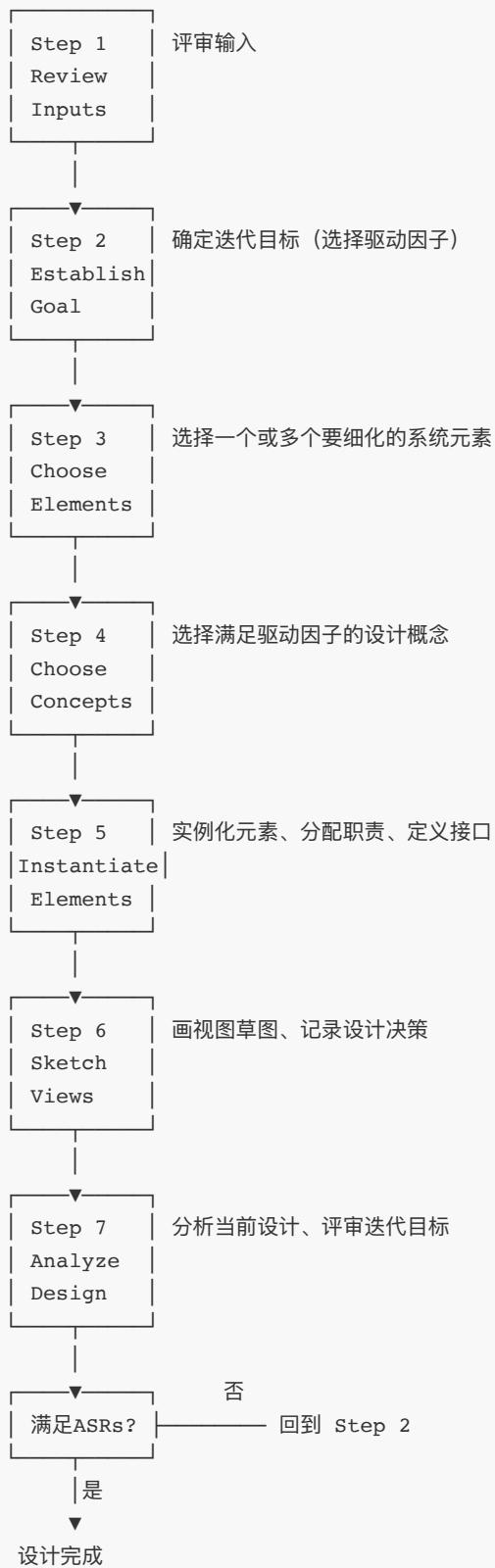
答题模板：

- 1. 定义战术和模式，强调粒度差异
- 2. 画关系图（模式包含多个战术）
- 3. 给2个模式例子，每个列出3-4个关联战术，并说明分别影响哪些质量属性

QI-5 架构设计主要过程步骤 (ADD 3.0)

ADD 3.0 的8个步骤

ADD (Attribute-Driven Design) 是一个迭代式的架构设计方法，核心思想是**让架构设计过程系统化、可追溯**。



每一步的输入和输出

步骤	名称	输入	输出	关键活动
Step 1	评审输入	设计目的、主要功能需求、质量属性场景、约束、关注点	经过验证和优先级排序的驱动因子列表	确保输入有效且优先级合理，避免"garbage in, garbage out"
Step 2	确定迭代目标	驱动因子及优先级	本轮迭代的目标声明（如"建立整体系统结构"或"满足QA-3可用性场景"）	选择本轮要处理的驱动因子子集，确保目标范围合理（不要太大也不要太小）
Step 3	选择要细化的元素	当前架构设计、迭代目标	被选中的元素（如整个系统、某个模块、某个层）	可做分解（top-down）、合并（bottom-up）或改进已有元素
Step 4	选择设计概念	迭代目标、被选中的元素、设计概念目录	选择的设计概念（参考架构、模式、战术、框架等）	通过优缺点表/SWOT/CBAM/原型等方法选择，避免重新发明轮子
Step 5	实例化元素并分配职责	选定的设计概念	新的架构元素及其职责和接口	将概念具体化为实际的系统元素（类、服务、组件等）
Step 6	画视图草图并记录决策	实例化的元素	架构视图草图和设计决策记录	这不是完整文档，而是草图级别的记录。用非正式符号即可，但要保持一致
Step 7	分析当前设计	当前设计、迭代目标、设计目的	设计看板更新、是否继续迭代的决定	使用设计看板跟踪进展，判断是继续迭代还是设计完成

设计终止条件

ADD迭代在以下任一条件满足时终止：

1. 所有ASRs的设计决策都已做出（**最佳**）
2. 最重要的技术风险已得到缓解
3. 分配给架构设计的时间已用完（**不推荐**）

 **我的理解：**ADD的核心哲学是 **"不要试图一次设计全部"**。每次迭代聚焦少量驱动因子，通过快速迭代逐步完善设计。这和敏捷思想是一致的——不是Big Design Up Front（BDUF），而是Just Enough Design。实际上：

- **Greenfield系统（成熟领域）：**建议先建整体结构（参考架构+部署模式），再支持主要功能，最后逐个处理质量属性
- **Greenfield系统（新领域）：**从原型和通用概念开始，因为没有现成的参考架构
- **Brownfield系统：**先理解现有架构，再用ADD迭代实现增量变更

答题模板：

1. 简介ADD是什么、为什么需要（让设计可系统化、可追溯）
2. 按顺序列出步骤，每步讲清楚输入和输出
3. 画一个简洁的流程图
4. 说明终止条件

QI-6 DDD中引入聚合模式的价值

背景：OO设计的局限性

在传统面向对象设计中，对象之间直接引用，形成一个复杂的对象网络。当业务逻辑复杂时：

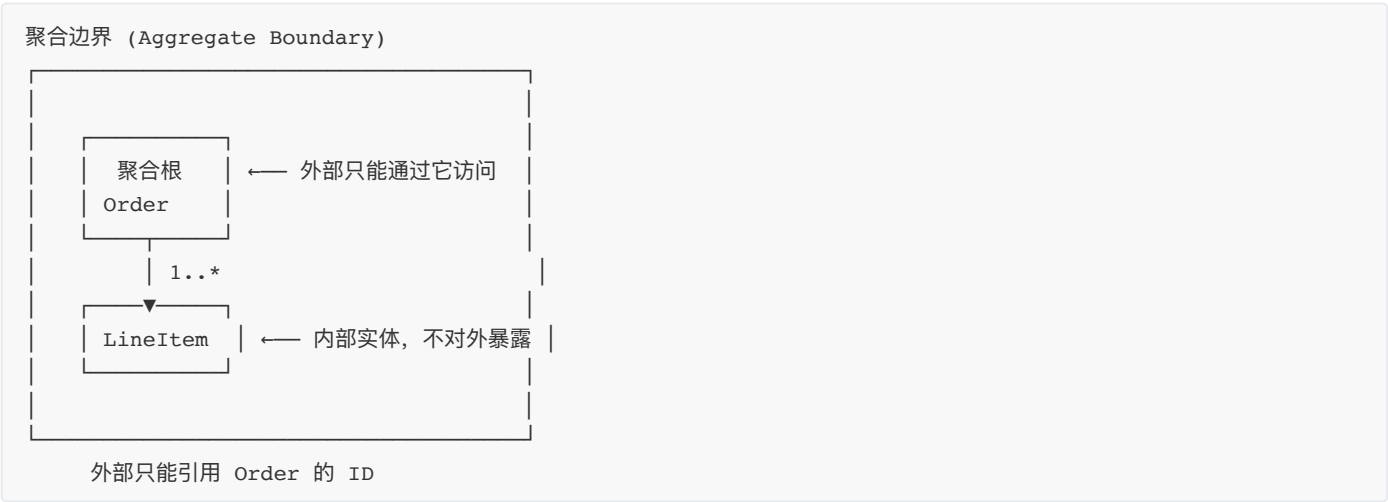
```
// 传统oo：任何对象都可以直接访问任何其他对象
order.getCustomer().getAccount().deduct(amount);    // 链条过长
order.getLineItems().get(0).setPrice(newPrice);      // 从外部直接修改内部
```

问题：

- 一致性边界模糊：哪些对象必须在一次事务中保持一致？不清楚
- 并发冲突：多个操作同时修改相关对象，容易出现数据不一致
- 业务规则分散：约束代码散落在各处，重复且容易遗漏
- 复杂度不可控：对象关系图变成一团乱麻

DDD聚合模式的定义

聚合（Aggregate）是一组相关对象的集合，以聚合根（Aggregate Root）为唯一入口，通过聚合根控制对内部对象的访问和操作。



聚合的核心价值

价值维度	OO设计	DDD聚合模式
事务一致性	不明确哪些对象需要一起修改	聚合是 事务边界 ：一个事务只修改一个聚合
不变量（Invariant）	业务规则散落，难以保证	聚合根负责保证聚合内所有不变量
并发控制	大范围锁或乐观锁冲突频繁	聚合越小，并发冲突越少
复杂度管理	对象图任意引用	通过ID引用其他聚合，保持边界清晰
演进性	牵一发而动全身	聚合内部修改不影响外部

聚合的设计原则

1. 小聚合优于大聚合：聚合足够小以提高性能（减少并发冲突），足够大以保证业务完整性
2. 通过ID引用其他聚合：不在聚合内部持有其他聚合的对象引用
3. 最终一致性处理跨聚合操作：使用领域事件实现跨聚合的最终一致性
4. 一个事务只修改一个聚合：这是聚合模式的黄金法则

 **我的理解：**聚合模式解决的其实是一个"边界"问题。传统OO设计中最大的问题是**没有一个明确的概念来界定"一致性边界"**。聚合把一个业务概念（如订单）和它内部紧密关联的对象（如订单行项目）打包在一起，对外只暴露一个入口（聚合根）。这就像公司的组织结构——你不能随便到每个员工工位上指挥，而是通过部门经理（聚合根）来协调。DDD的其他战术模式（实体、值对象、领域服务、仓库、工厂）都围绕聚合来组织，聚合是DDD战术设计的核心单元。

答题模板：

1. 先指出OO设计的问题（一致性边界模糊、对象图复杂）
2. 定义聚合模式（一组对象 + 聚合根 + 事务边界）
3. 分析4-5个方面的价值（事务边界、不变量、并发、复杂度、演进性）
4. 给出代码对比示例

QI-7 为什么需要不同视图 + 4个示例视图

为什么需要不同视图？

核心原理：软件架构是一个复杂的多维人工制品，没有任何单一的表示方式能完整地、无歧义地描述它。

类比建筑学：一栋建筑需要**平面图、立面图、结构图、电气图、管道图**等不同图纸，每种图纸服务于不同的涉众（业主关心平面布局，结构工程师关心承重墙，电工关心布线）。

四个具体原因：

1. **不同涉众关心不同方面：**开发者关心模块接口，运维关心部署拓扑，项目经理关心工作分配，安全审计员关心数据流
2. **不同质量属性需要不同视角来推理：**性能需要C&C视图（看运行时交互），可修改性需要模块视图（看代码依赖），安全性需要分配视图（看物理部署）
3. **单视图无法同时表达结构和行为：**静态的模块分解 $\neq$ 运行时的组件交互
4. **控制复杂度和信息过载：**把信息分解到不同视图中，每个视图聚焦一个维度

四大类视图（架构视图体系）

视图类型	视图名称	元素	关系	目的	主要涉众
模块视图	分解视图 (Decomposition)	模块 (实现单元)	is-part-of (整体-部分)	展示代码组织方式, 支持工作分配、变更影响分析	开发人员、项目经理
模块视图	使用视图 (Uses)	模块	depends-on (依赖)	展示模块间依赖关系, 支持增量开发、需求追溯	开发人员
模块视图	分层视图 (Layered)	层 (Layer)	allowed-to-use (允许使用)	控制模块间的可见性和依赖方向	架构师、开发人员
C&C视图	客户-服务器视图	客户端、服务器、连接器	attachment (附着)	展示运行时组件如何交互, 支持性能和可用性推理	架构师、运维人员
C&C视图	SOA视图	服务、ESB	服务调用/编排	展示企业级服务集成	架构师、集成工程师
分配视图	部署视图 (Deployment)	软件元素、硬件节点	allocated-to (部署到)	展示软件到硬件的映射, 支持性能 and 安全性推理	运维人员、安全工程师
分配视图	工作分配视图 (Work Assignment)	模块、团队	assigned-to (分配给)	展示模块到开发团队的分配	项目经理
质量视图	安全视图 (Security)	安全相关组件	安全通信	聚焦安全组件和威胁应对	安全分析师

🧠 我的理解：视图选择的核心准则是 "按需选视图" — 你要回答什么问题，就选什么视图。至少要有1个模块视图、1个C&C视图，大系统还需要1个分配视图。对于考试，建议记住4个最常用的视图名称和目的：**分解视图**（代码组织）、**分层视图**（依赖管理）、**C&C视图**（运行时交互）、**部署视图**（物理部署）。

答题模板：

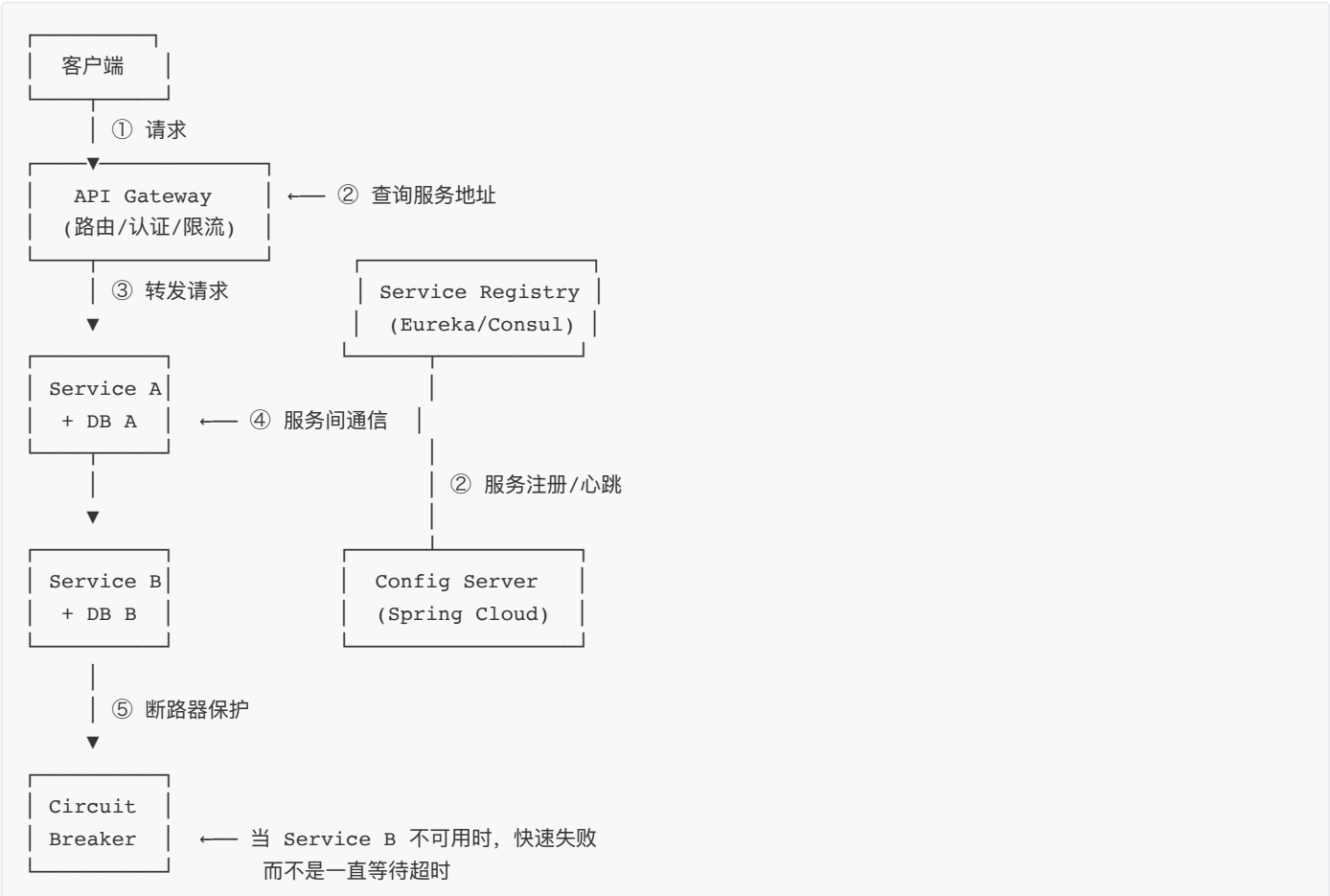
1. 先答"为什么要多视图" (4个原因)
2. 列出4个视图，每个给出：名称、元素类型、关系类型、目的、使用场景
3. 指出"最少1个模块视图 + 1个C&C视图；大系统加1个分配视图"

QI-8 微服务系统的4个架构元素

四个核心架构元素

#	架构元素	英文	角色	类比
1	API网关	API Gateway	统一的入口：路由请求、认证鉴权、限流、请求聚合、协议转换。所有外部请求必须先经过网关	写字楼的前台/大堂，所有访客从这进出
2	服务注册与发现	Service Registry & Discovery	动态地址簿：服务启动时注册自己的地址，调用方通过注册中心发现服务实例位置	公司的内部分机号簿，员工换座位后更新号码
3	服务（业务能力）	Service (Business Capability)	业务能力的自治单元：每个服务围绕一块业务能力构建，拥有独立的数据、独立的团队、独立的发布周期	公司里独立的部门（财务部、人事部、研发部）
4	配置管理/断路器	Config Server / Circuit Breaker	韧性基础设施：集中管理各服务的配置；断路器防止级联故障，当某个服务不可用时快速失败，保护系统整体	保险丝，一个房间短路不影响整栋楼

交互关系



🧠 我的理解：微服务不是"把服务做小"那么简单，它必须配套这4个基础设施元素才能运转。如果没有API网关，客户端就要知道所有服务的地址；没有服务发现，服务实例的动态伸缩就无从谈起；没有断路器，一个服务挂了就会拖垮整个系统。所以微服务的核心挑战不是"如何拆分"，而是"如何让分布式的多个服务像一个整体一样协作"。

答题模板：

- 1. 列出4个元素，每个给出名称和角色
- 2. 画一张简化的交互关系图

事件驱动架构（Event-Driven Architecture, EDA）的核心思想是：系统各组件通过发布和订阅"业务事实（事件）"进行异步协作，而不是通过同步的请求-响应调用。

传统同步模式：

Service A —调用→ Service B —调用→ Service C
(A必须等B, B必须等C, 链条越长越脆弱)

事件驱动模式：

```

graph LR
    P[Producer] -- "发布事件" --> E[Event Bus/Broker]
    E -- "订阅" --> CA[Consumer A]
    E -- "订阅" --> CB[Consumer B]

```

(生产者和消费者互不知道对方存在，通过事件总线解耦)

- **生产者**发布事件（业务事实，如"订单已创建"），不关心谁来消费
- **消费者**订阅感兴趣的事件，独立处理
- **事件总线/Broker**（如Kafka、RabbitMQ）负责事件的持久化、路由和分发
- **事件是过去时的事实**：不是"创建订单"，而是"订单已创建"

质量属性	影响	原因
可扩展性 ↑	↑ 显著提升	新增消费者不影响现有系统，只需订阅新的事件类型即可
可用性/韧性 ↑	↑ 显著提升	消费者宕机不影响生产者；消息可持久化，消费者恢复后继续处理
弹性 ↑	↑ 显著提升	事件总线天然支持削峰填谷，流量波峰被消息队列吸收
可修改性 ↑	↑ 提升	生产者和消费者解耦，修改一个不影响另一个
性能（吞吐）↑	↑ 提升	异步处理提升整体吞吐量，不阻塞调用方
可调试性 ↓	↓ 下降	异步链路难以追踪，时序不确定，排查问题需要分布式追踪
可理解性 ↓	↓ 下降	系统的端到端流程分散在事件链中，没有集中的流程视图
强一致性 ↓	↓ 下降	事件处理是异步的，只能保证最终一致性，无法做同步事务
时序复杂性 ↓	↓ 下降	事件的到达顺序和处理顺序可能不一致，需要处理乱序和幂等

🧠 **我的理解：**事件驱动不是银弹。它本质上是用 "理解成本的上升" 换 "运行弹性的提升"。什么时候该用？看一个简单判断：如果你的系统天然就是 "发生了一件事，然后多个地方需要反应"（如订单创建后要发短信、减库存、记日志），事件驱动就是最自然的表达方式。但如果你的业务需要强一致性的同步确认（如支付扣款），就不要硬套事件驱动。

1. 解释EDA基本思想（发布-订阅、异步解耦、事件总线）
2. 画一个简单的对比图（同步调用 vs 事件驱动）

- 3. 列出质量属性权衡表（至少5个提升 + 4个牺牲）
- 4. 结论： 适合"一对多反应"的场景， 不适合强一致性场景

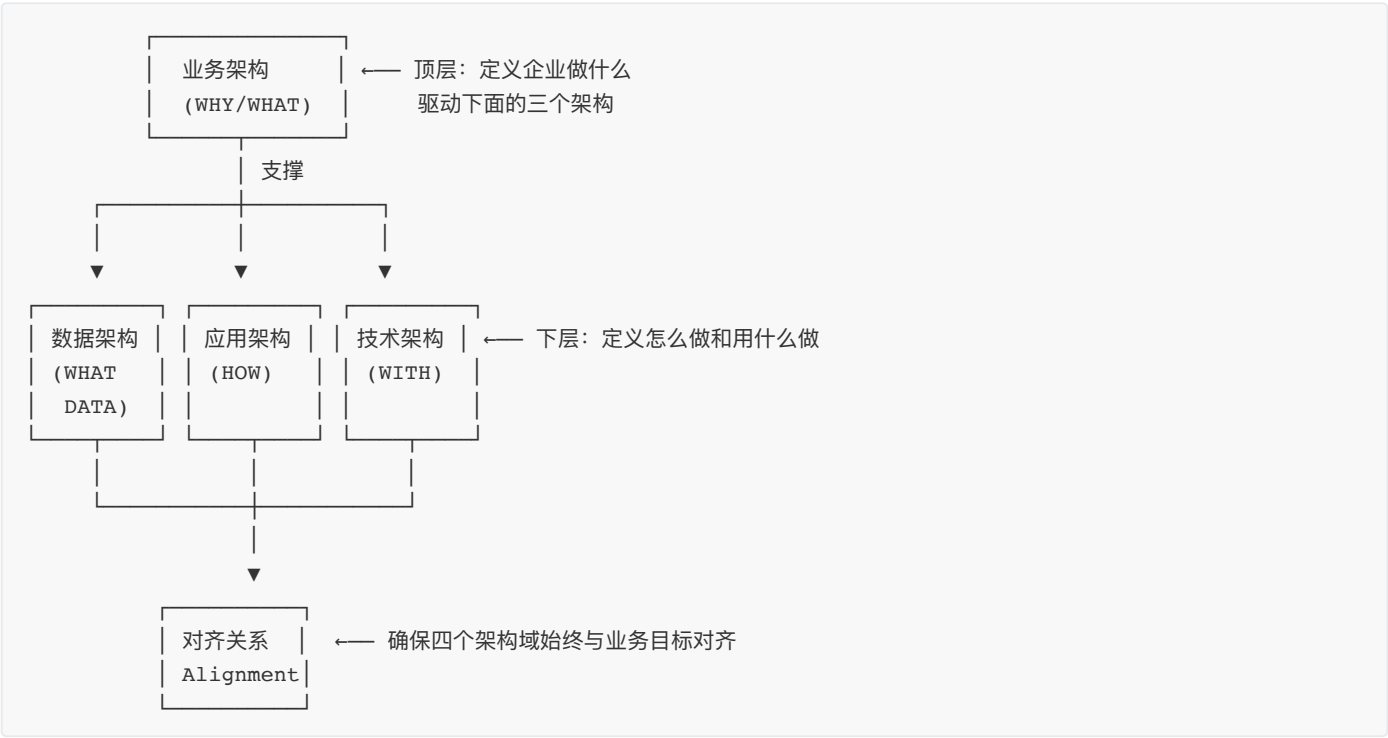
QI-10 企业架构中的4A

概念定义

企业架构（Enterprise Architecture, EA）是对企业IT资产的**全景式描述**，确保IT投资与业务战略对齐。4A是EA的四个核心架构域。

架构域	英文	核心问题	关键内容	产出物
业务架构	Business Architecture	干什么？谁来做？	业务战略、业务流程、组织架构、业务能力地图	业务流程图、组织结构图、业务能力模型
应用架构	Application Architecture	用什么系统来支撑？	应用系统及其功能、应用间交互、应用与业务的对应关系	应用系统清单、应用集成图、应用功能地图
数据架构	Data Architecture	数据怎么存、怎么管？	数据模型、数据分布、数据治理、主数据管理	数据模型图、数据流向图、数据资产目录
技术架构	Technology Architecture	跑在什么上面？	硬件设施、中间件、网络、云平台、DevOps工具链	技术平台架构图、基础设施拓扑图

4A之间的关系



核心关系：

- 1. 业务架构是其他三个架构的"客户"：应用、数据、技术架构服务于业务架构
- 2. 应用架构依赖数据架构和技术架构：应用需要数据和技术平台的支撑
- 3. 数据架构和技术架构相互约束：技术的存储能力限制了数据方案的选择，数据规模反过来影响技术选型

4. 四个架构域必须保持对齐：业务变化 → 应用调整 → 数据重构 → 技术升级，任何一层的变化需要其他层同步调整

 我的理解：4A本质上是把"企业IT"从四个维度拆开看。传统的问题是业务部门和技术部门各自为政——业务说要这个功能，技术说做不了。4A框架强迫你在做任何技术决策时都要回答：这支持了什么业务目标？数据怎么管？需要什么应用？底层用什么技术？四个问题同步思考，才不会造出"技术上很酷但业务上没用"的系统。

答题模板：

- 1. 介绍企业架构的定义和目的
- 2. 列出4A，每个给出核心问题和关键产出物
- 3. 画4A关系图，强调业务架构是顶层驱动
- 4. 总结：4A确保IT与业务对齐

Part II：分析论述题（2题×10分=20分）

QII-1 七类设计决定与绑定时间

七类设计决定

设计决定（Design Decisions）是架构设计过程的核心产物。ADD的每一步本质都是在做设计决定。

#	类别	英文	说明	例子
1	职责分配	Allocation of Responsibilities	将系统职责分解并分配给各架构元素	认证职责分配给Auth Service，订单处理分配给Order Service
2	协调模型	Coordination Model	架构元素之间如何通信和协作	同步REST调用 vs 异步消息 vs RPC
3	数据模型	Data Model	数据的组织方式、存储策略和访问模式	关系型 vs NoSQL；每服务独立数据库 vs 共享数据库
4	资源管理	Management of Resources	如何管理CPU、内存、网络、存储等资源	连接池、缓存策略、限流机制、数据库连接管理
5	架构元素间的映射	Mapping Among Architecture Elements	架构元素如何映射到软件实现、团队和物理节点	模块 → 代码包；服务 → Docker容器；模块 → 开发团队
6	绑定时间决策	Binding Time Decisions	系统变化在什么时间点之前可以被确定	编译时绑定 vs 部署时绑定 vs 运行时绑定
7	技术选择	Choice of Technology	选择具体的技术栈和工具	Spring Boot vs Go；MySQL vs PostgreSQL；Kafka vs RabbitMQ

 我的理解：这七类决策是有先后顺序的。一般来说，先做职责分配（把系统拆开），然后决定协调模型（各部分怎么通信），再做数据模型（数据怎么管），最后才是技术选择。如果一上来就选技术栈（"我们用Kubernetes + Kafka！"），那是本末倒置——你还没搞清楚系统需要什么呢。

绑定时间（Binding Time）

定义：绑定时间指的是系统中的一个可变决策可以在什么时间点之前被确定/固定下来。在这个时间点之前系统保持灵活性，之后便不可更改。

绑定时间的几种选择



绑定时间对质量属性的影响

绑定时间	可修改性	性能	可部署性	可测试性
编码时	↓↓ 最低	↑↑ 最高	↓↓ 差	↓ 差
编译时	↓ 较低	↑↑ 高	↓ 较差	↓ 较差
部署时	↑ 高	↑ 较高	↑↑ 好	↑ 好
运行时	↑↑ 最高	↓ 较低	↑↑ 最好	↓ 复杂

核心权衡：

- 越晚绑定 → 灵活性越高 → 可修改性越好
- 越早绑定 → 确定性越高 → 性能越好、越容易测试
- 部署时绑定是大多数场景的最佳平衡点（配置文件方式）

 我的理解：绑定时间的选择本质上是在用"当下的确定性"换"未来的灵活性"。一个经典案例就是微服务的服务发现——如果服务的地址在编译时就写死，性能最好但改个地址就要重新部署；如果用Consul/Eureka做运行时发现，灵活但每次调用都要查一次注册中心。具体选哪个绑定时间，取决于你的系统对这个可变点的变化频率和性能敏感度。

答题模板：

1. 列出7类设计决定（建议列表格）
2. 定义绑定时间，列出4-5种选择（从编码时到运行时）
3. 分析不同绑定时间对质量属性的影响（用表格）
4. 总结权衡：晚绑定 = 灵活性 ↑，性能 ↓；早绑定 = 性能 ↑，灵活性 ↓

这是2026年考试的分析论述题，分值10分，要求"选择具体系统场景"来论述。建议选择企业级电商/订单系统或选课系统作为场景。

场景设定：企业电商平台

假设我们从1990年代开始建设一个企业级电商平台，经历了三代架构演进。

第一代：C/S 架构（1990s）

时代约束：PC和GUI普及，用户需要富桌面交互



解决了什么问题？

- ✔ 从主机/终端时代迈入桌面交互时代，用户体验大幅提升
- ✔ 部分逻辑在客户端本地执行，响应更快
- ✔ 部门级应用可以快速上线

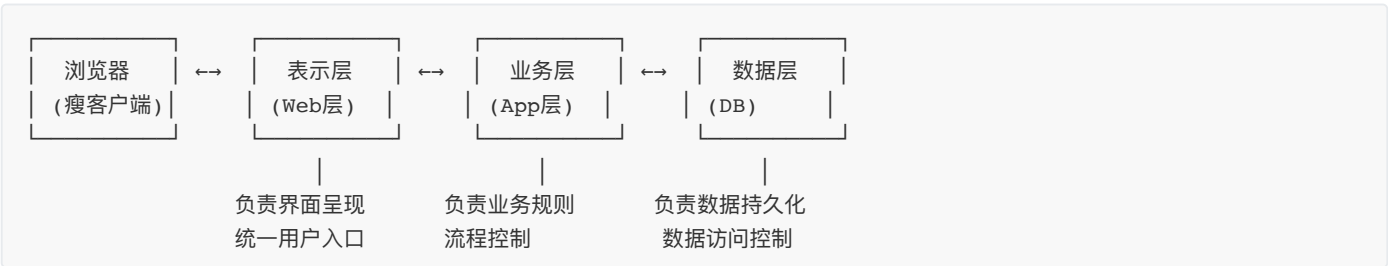
产生了什么局限？

- ✘ 部署噩梦：每个客户端都要安装、升级，1000台电脑=1000次安装
- ✘ 版本混乱：不同客户端运行不同版本，兼容性问题严重
- ✘ 客户端与数据直连：业务逻辑和安全校验分散在客户端，难以统一治理
- ✘ 可维护性差：业务规则变更需要更新所有客户端

🔗 主要矛盾：把"交互体验"做好了，但把"部署和治理"搞砸了

第二代：三层/分层架构（2000s）

时代约束：Web浏览器普及，需要统一入口并降低维护成本



解决了什么新问题？

- ✔ 部署成本大幅下降：浏览器统一接入，无需安装客户端
- ✔ 系统内部职责清晰：表示层变化（改UI）、业务层变化（改规则）、数据层变化（改存储）互不影响
- ✔ 安全更可控：数据访问不暴露到客户端
- ✔ 可维护性大幅提升：业务规则集中管理，统一修改

产生了什么新局限？

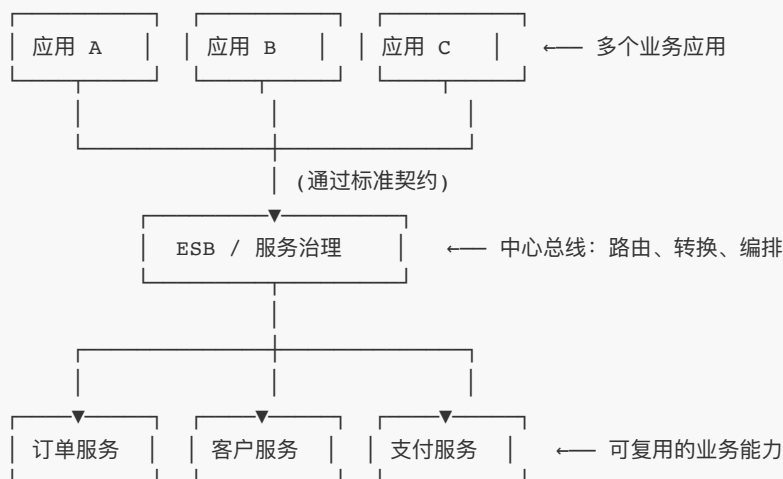
- ✘ 系统间割裂：不同业务系统仍然是孤岛（订单系统、库存系统、支付系统各自为政）
- ✘ 能力重复建设：多个系统各自实现"用户认证"、"订单查询"等相同能力

- ❌ **端到端流程断裂**：一个完整的采购流程横跨多个系统，靠大量点对点接口连接
- ❌ **复杂度堆积在单应用内部**：随着业务增长，单应用越来越庞大（"大泥球"问题）

🌀 **主要矛盾**：把"单个系统内部"整清楚了，但"系统之间"还是混乱的

第三代：SOA 架构（2000s后期）

时代约束：企业内系统数量激增（ERP、CRM、SCM、OA...），需要跨系统复用能力和打通端到端业务流程



解决了什么新问题？

- ✅ **跨系统能力复用**：订单服务可以被订单系统、客服系统、报表系统共同使用
- ✅ **流程编排**：通过ESB/BPEL显式编排端到端业务流程
- ✅ **异构整合**：不同技术栈的系统可以进入统一的协作体系
- ✅ **契约标准化**：通过WSDL/SOAP等标准化接口契约实现服务解耦
- ✅ **企业级治理**：统一的服务注册、版本管理、权限和安全策略

产生了什么新局限？

- ❌ **中心治理瓶颈**：ESB成为变更瓶颈，所有服务集成都要经过中心团队
- ❌ **服务粒度偏粗**：一个"订单服务"可能包含查询、创建、支付、退款等全部逻辑
- ❌ **重协议**：SOAP/WSDL等协议复杂，开发和调试成本高
- ❌ **跨团队依赖**：发布一个端到端功能需要多个团队协调
- ❌ **重量级基础设施**：ESB、注册中心、编排引擎等中心组件本身成为复杂度的新来源
- ❌ **不适合互联网快速迭代**：中心治理与高频发布矛盾尖锐

🌀 **主要矛盾**：把"企业级协同"做成了，但"团队自治和快速演进"被牺牲了
 （这也正是微服务架构随后出现的原因——用更小的服务粒度、更轻的契约和去中心化的治理来换发布速度）

演进逻辑总结

C/S 架构

主要矛盾：要把交互做到桌面端

优先：易用性、响应性

牺牲：可部署性、可维护性

↓ 新约束：Web统一入口；部署成本爆炸

三层/分层架构

- 主要矛盾：要把系统内职责分清、统一入口
- 优先：可维护性、可修改性、可部署性
- 牺牲：极致性能、跨系统协同
- ↓ 新约束：系统数量激增；需要跨系统协同

SOA 架构

- 主要矛盾：要跨系统复用能力、打通流程
- 优先：互操作性、可复用性、可治理性
- 牺牲：简单性、轻量部署、局部演进速度

🧠 **核心洞察**：这不是"哪个架构更好"的问题，而是**每个阶段的主要矛盾不同**。C/S解决的是"交互体验"问题，三层解决的是"单系统治理"问题，SOA解决的是"跨系统协同"问题。如果你今天只是做一个部门级小系统，三层/分层可能就是最好的选择——不需要SOA的重治理，也不需要微服务的分布式复杂度。**架构选择永远要回答"我的系统今天到底卡在哪一层矛盾上"**。

答题模板：

- 选择场景**：明确描述一个具体系统（建议用电商订单系统或选课系统）
- C/S → 三层**：
 - C/S解决了什么（富交互）→ 产生了什么问题（部署噩梦）
 - 三层解决了什么（统一入口+职责分层）→ 是怎么解决的
- 三层 → SOA**：
 - 三层遗留了什么问题（系统间割裂）→ 业务发展暴露出什么新矛盾
 - SOA解决了什么（跨系统复用+流程编排）→ 是怎么解决的
- 总结**：架构演进的核心是"主要矛盾的迁移"，不是"新架构替代旧架构"
- 加分点**：指出每种架构的质量属性取舍（优先了什么、牺牲了什么）

Part III：设计题（20分）

DDD事件风暴 + 聚合设计方法

2026年考试设计题场景：实验耗材申领与采购平台
核心流程：申请 → 审批 → 采购 → 入库

方法论：事件风暴（Event Storming）

事件风暴是DDD中用于探索复杂业务领域的协作建模方法。核心步骤：

- | | |
|--------------------------------|------------------|
| Step 1：领域事件（Domain Events） | → 识别"发生了什么" |
| Step 2：命令（Commands） | → 识别"谁触发了什么" |
| Step 3：聚合（Aggregates） | → 识别"数据的一致性边界" |
| Step 4：策略/流程（Policies/Process） | → 识别"业务规则和自动化流程" |
| Step 5：外部系统 & 读模型 | → 识别"依赖和展示" |

关键考点：事件命名规则

⚠️ 事件使用过去时业务事实，不写UI操作！

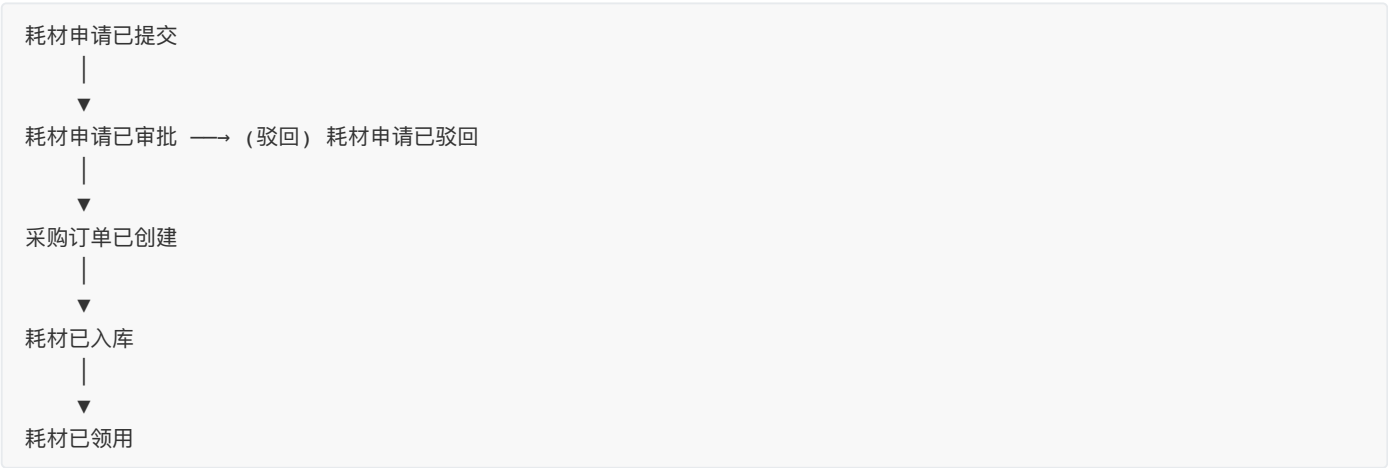
❌ 错误写法（UI视角）	✅ 正确写法（业务事实）
用户点击申请按钮	耗材申请已提交
管理员审批通过	耗材申请已审批
系统下单采购	采购订单已创建
仓库确认收货	耗材已入库

考试场景：实验耗材申领与采购平台

业务流程还原：

- 1. 研究人员提交耗材申请
- 2. 实验室主管审批申请（通过或驳回）
- 3. 审批通过后，采购员创建采购订单
- 4. 供应商发货，仓库管理员确认入库
- 5. 研究人员领用耗材

Step 1: 领域事件流



Step 2: 识别命令和聚合

领域事件	触发命令	聚合	聚合根
耗材申请已提交	提交耗材申请	耗材申请（Requisition）	Requisition
耗材申请已审批	审批耗材申请	耗材申请（Requisition）	Requisition
采购订单已创建	创建采购订单	采购订单（PurchaseOrder）	PurchaseOrder
耗材已入库	确认入库	库存（Inventory）	InventoryItem
耗材已领用	领用耗材	库存（Inventory）	InventoryItem

Step 3: 聚合设计和不变量

聚合1: Requisition（耗材申请）

Requisition（聚合根）

- id: RequisitionId

← 值对象

1. 只能审批"待审批"状态的申请
2. 驳回后不可再次审批同一申请
3. 申请必须包含至少一个耗材项

PurchaseOrder (聚合根)

← 通过ID引用其他聚合

1. 只能基于"已审批"的申请创建
2. 采购总金额不能超过预算限制

InventoryItem (聚合根)

1. 库存数量不能为负
2. 领用数量不能超过库存数量

策略	触发事件	动作
自动审批策略	耗材申请已提交	如果金额 < 500元，自动审批（无需人工）
库存预警策略	耗材已领用	如果库存 < 安全阈值，自动发出补货提醒
采购完成策略	耗材已入库	更新申请状态为"已完成"

1. **聚合要小**：不要把 Requisition 和 PurchaseOrder 合并为一个聚合。它们有不同的生命周期和一致性需求，通过ID引用即可
2. **跨聚合用事件**：当采购入库后需要更新申请状态时，通过"耗材已入库"事件触发，而不是在入库事务中直接修改申请
3. **一个事务一个聚合**：每个事务只修改一个聚合。如果需要修改多个聚合，使用最终一致性+补偿机制
4. **事件命名用业务语言**：不是"点击按钮"，而是"申请已提交"——让业务人员和开发人员用同一种语言交流

答题模板：

- 1. **事件流**：按业务时序画出关键领域事件（5-7个），用过去时命名
- 2. **聚合识别**：列出主要聚合，每个包含关键属性和不变量
- 3. **策略**：列出1-2个自动化策略（如自动审批、库存预警）
- 4. **注意**：说明聚合间的ID引用和最终一致性

附录：质量属性速查表

常见质量属性分类

类别	质量属性	核心关注	典型战术
运行时（外部可观察）	性能（Performance）	系统多快？	缓存、并行处理、减少通信、资源调度
	可用性（Availability）	系统什么时候能用？	冗余、故障检测、故障恢复
	安全性（Security）	谁能访问？数据安全吗？	加密、认证、授权、审计
	易用性（Usability）	系统好用吗？	用户建模、反馈机制、取消操作
	互操作性（Interoperability）	能和其他系统协作吗？	标准接口、数据转换、服务编排
	可靠性（Reliability）	结果能信赖吗？	数据校验、事务、投票机制
开发时（内部不可见）	可修改性（Modifiability）	改起来容易吗？	封装、分层、依赖注入、推迟绑定
	可测试性（Testability）	好测试吗？	依赖注入、接口抽象、环境隔离
	可复用性（Reusability）	能拿给别人用吗？	通用接口、松耦合、组件化
	可移植性（Portability）	能跑在不同环境吗？	抽象平台差异、标准化接口

质量属性场景6要素速查



考试策略建议

时间分配

部分	题目数	分值	建议用时
Part I 简答题	10题	60分	70-80分钟（每题7-8分钟）
Part II 分析论述题	2题	20分	30-35分钟（每题15-17分钟）
Part III 设计题	1题	20分	30-35分钟
检查	—	—	5-10分钟

答题策略

- 1. 简答题：概念定义 → 关系/区别 → 例子或权衡。宁可多写不要少写，但每点要准确
- 2. 分析论述题：先给出整体框架，再展开论述，最后总结。QII-2架构演进题一定要写出"主要矛盾迁移"的逻辑
- 3. 设计题：事件命名用过去时！聚合要合理（不大不小），每个聚合列出不变量
- 4. 英文题目、中文作答：先看懂英文题面，用中文组织答案会更高效

高频易错点

- 1. ❌ 架构 = 盒式图 → ✅ 架构包含结构、元素、关系和行为
- 2. ❌ 质量属性是需求之外的东西 → ✅ 质量属性是需求的一部分，是ASR的主要来源
- 3. ❌ 模式比战术高级 → ✅ 模式由战术组成，模式解决整体问题，战术解决局部问题
- 4. ❌ 架构越新越好 → ✅ 架构选型取决于主要矛盾，分层单体也可能是最优解
- 5. ❌ 事件写"点击按钮" → ✅ 事件写"订单已创建"（过去时业务事实）
- 6. ❌ 绑定时间越晚越好 → ✅ 越晚越灵活但有性能成本，部署时绑定是最佳平衡点

延伸阅读：

- *Software Architecture in Practice* (4th ed.) — Bass, Clements, Kazman
- *Fundamentals of Software Architecture* — Mark Richards, Neal Ford
- *Building Microservices* (2nd ed.) — Sam Newman
- *Domain-Driven Design* — Eric Evans

参考来源：

- [NJU软件体系结构2026考试回忆卷](#)
- [CSDN: 南大软件体系结构期末复习](#)
- [EagleBear2002 质量属性笔记](#)
- 课程课件：ADD 3.0（李杉杉）、软件架构模式（吕骏）

Last updated: 2026/06/08

祝考试顺利，拿下高分！🎓